HDOC Day-60 Activity

Anagram Check Using Character Frequency Counting and Pointers

Student Name	Dhruv Pandey
SAP ID	590037660
Batch	15
Course / Activity	HDOC Day-60

Aim

To write a C program that checks whether two lowercase strings are anagrams by comparing the frequency of every character. Both strings are passed to **check_anagram()** using their addresses and accessed using pointers.

Problem Requirements

- Read two lowercase strings, **s** and **t**.
- Pass both strings to **check_anagram()** using their addresses.
- Check whether both strings contain the same characters with the same frequencies.
- Ignore the order of characters.
- Print **Anagram** when they are valid anagrams.
- Print **Not Anagram** when they are not anagrams.
- Strings having different lengths must be treated as not anagrams.
- Use a character-frequency-counting approach.

Algorithm

1. Read strings s and t.
2. Pass their addresses to **check_anagram(s, t)**.
3. Compare their lengths. If the lengths differ, print Not Anagram and return.
4. Create an integer frequency array **freq[26]** and initialize all values to 0.
5. Traverse s and increment the frequency of each character using **freq[*(s+i)-'a']++**.
6. Traverse t and decrement the frequency of each character using **freq[*(t+i)-'a']--**.
7. If every value in freq[] is 0, both strings contain identical character counts and are anagrams.
8. Otherwise, print Not Anagram.

C Program

```
#include <stdio.h>
#include <string.h>

void check_anagram(char *s, char *t)
{
    int len1 = strlen(s);
    int len2 = strlen(t);

    if (len1 != len2)
    {
        printf("Not Anagram");
        return;
    }

    int freq[26] = {0};

    for (int i = 0; i < len1; i++)
    {
        freq[*(s + i) - 'a']++;
    }

    for (int i = 0; i < len2; i++)
    {
        freq[*(t + i) - 'a']--;
    }

    for (int i = 0; i < 26; i++)
    {
        if (freq[i] != 0)
        {
            printf("Not Anagram");
            return;
        }
    }

    printf("Anagram");
}

int main()
{
    char s[1000], t[1000];

    printf("Enter string s: ");
    scanf("%999s", s);

    printf("Enter string t: ");
    scanf("%999s", t);

    check_anagram(s, t);

    return 0;
}
```

Pointer Usage

check_anagram(s, t) passes the addresses of the first characters of both strings. The parameters **char *s** and **char *t** receive those addresses. ***(s+i)** and ***(t+i)** access individual characters.

Frequency Counting - Example 1

Character	Frequency in listen	Frequency in silent
e	1	1

i	1	1
l	1	1
n	1	1
s	1	1
t	1	1

Both strings contain exactly the same characters with the same frequencies. Therefore, **listen** and **silent** are anagrams.

Test Cases / Expected Results

Case	String s	String t	Expected Output
Example 1	listen	silent	Anagram
Example 2	hello	world	Not Anagram
Example 3	anagram	nagaram	Anagram
Example 4	rat	car	Not Anagram

How Frequency Counting Works

The frequency array has 26 positions, one for each lowercase English letter. While reading the first string, counts are increased. While reading the second string, the same counts are decreased. If the strings contain exactly the same letters in the same quantities, every frequency returns to zero.

Different Lengths

Two strings of different lengths cannot contain exactly the same number of characters. Therefore, the program immediately prints **Not Anagram** when their lengths are different.

Complexity

Time Complexity: O(n), where n is the string length, because each string is traversed once and the fixed 26-element frequency array is checked once.

Space Complexity: O(1), because freq[] always contains exactly 26 integers regardless of input size.

Result

The program successfully determines whether two lowercase strings are anagrams using pointer-based string access and character-frequency counting. Character order is ignored, while character frequencies and string lengths are correctly considered.

Execution Example

```
Enter string s: listen
Enter string t: silent
Anagram
```